

CODE GNAN

NUTRIFY WEB APPLICATION



My details

Name : Sanaka Nadeesh

College name : Indian Institute of Technology, Goa

Currently pursuing : Second year, Bachelor of Technology, IIT Goa

Abstract

A growing body of research proves that fruits are critical to promoting good health. In fact, fruits and vegetables should be the foundation of a healthy diet. Most people need to double the amount of fruits and vegetables they eat every day.

Fruits supply dietary fibre and fibre intake is linked to lower incidence of cardiovascular disease and obesity. Fruits and vegetables also supply vitamins and minerals to the diet and are sources of phytochemicals that function as antioxidants, phytoestrogens, and antiinflammatory agents and through other protective mechanisms.

In this scenario, it is not only important to consume the right amount of fruits but also it is important to the nutritional value of fruits we consume.

The project 'Nurify Web Application' is a web-based application that gives a user the nutritional information of fruit when fruit name is entered.

Implementation Overview

First approach :

The implementation was of two main stages :

1. Data analysis
2. Web application building

1. Data analysis :

The data set was a huge 14164 x 117 table of which only some rows and columns were needed.

After minimizing the data to the form as needed, the code was written such that when the name of a fruit was given as input, it formed a piechart.png in a specified

directory. Each time the code was run, the piechart.png was replaced with a new pie chart.

2. Web application building :

The basic idea of the website was when a string like '/Apples' is given in the URL, the data analysis code should run and the piechart.png should be displayed.

Firstly I completely understood and implemented the whole web application using Django without the above data analysis code involved in it and then added the pie chart part to it.

Error:

On adding data analysis part to the web application part. There was an error NSException and cache issue, i.e, without clearing browser cache manually, piechart.png was not getting updated.

Second approach :

After trying various methods to solve the above errors, the following idea was implemented.

The implementation was of two main stages :

1. Data analysis
2. Web application building

1. Data analysis :

The data set was a huge 14164 x 117 table of which only some rows and columns were needed.

After minimizing the data to the form as needed, the code was written to form pie charts for each of the fruit in the data set and store it in a directory. Storing the pie charts data removed both the problems of NS exception as well as cache problems.

2. Web application building :

Develop web application with simple user interface using flask.

Implementation

Dataset Description

One of the most difficult issues while starting this project was to get the correct data set in the correct organisation.

The data set was a 14164 x 117 .xlsx table.

Categories: There were 24 different categories of food in the data set.

Ex: Restaurant Foods, Meats, Nuts and Seeds, Fish, Breakfast Cereals, Vegetables, Dairy and Egg Products, Fruits, Beans and Lentils, etc.

The data set had detailed information about food that belonged to any of the above categories. The information included the average percentage of Vitamin A, Vitamin B, Vitamin E, Carbohydrates, Iron, Calcium, Sodium, Magnesium, etc. It had information about around 100 such components of food.

Data analysis using Python

The data set was minimized for better visualization and understanding of the vital components of the fruits.

All the categories other than fruits were removed and all the components which were in very less proportion like the content of Riboflavin in fruits were also removed.

For analysing the data in the above-described data set, few libraries were installed from the command prompt and they have been imported to the data analysis code, data.py

Following is the code snippet from data.py to import libraries :

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import matplotlib
```

Data set minimisation :

Minimizing the 14164 x 117 table :

The following is the code snippet that implements the data set/table minimisation :

```
matplotlib.rcParams['axes.labelsize'] = 14

matplotlib.rcParams['xtick.labelsize'] = 12

matplotlib.rcParams['ytick.labelsize'] = 12

matplotlib.rcParams['text.color'] = 'k'

dif = pd.read_excel("/Users/supriyauppala/Desktop/myflask/Mydata.xlsx")

#fruits = dif.loc[dif['Food Group'] == 'Fruits']

df = dif.loc[dif['Food Group'] == 'Fruits']

#vegetables = dif.loc[dif['Food Group'] == 'Vegetables']

#df = pd.concat([fruits, vegetables], ignore_index=True)
```

```
# df=pd.concat([fruits_and_vegetables, beans_and_lentils], ignore_index=True)

df.drop(["ID", "Calcium (mg)", "Iron, Fe (mg)", "Potassium, K (mg)",
"Magnesium (mg)", "Vitamin A, IU (IU)", "Vitamin A, RAE (mcg)",
"Vitamin C (mg)", "Vitamin B-12 (mcg)", "Vitamin D (mcg)",
"Vitamin E (Alpha-Tocopherol) (mg)", "Added Sugar (g)", "Net-Carbs (g)",
"Omega 3s (mg)", "Omega 6s (mg)", "PRAL score", "Trans Fatty Acids (g)",
"Soluble Fiber (g)", "Insoluble Fiber (g)", "Sucrose (g)", "Glucose (Dextrose)
(g)",
"Fructose (g)", "Lactose (g)", "Maltose (g)", "Galactose (g)", "Starch (g)",
"Total sugar alcohols (g)", "Phosphorus, P (mg)", "Sodium (mg)", "Zinc, Zn (mg)",
"Copper, Cu (mg)", "Manganese (mg)", "Selenium, Se (mcg)", "Fluoride, F (mcg)",
"Molybdenum (mcg)", "Chlorine (mg)", "Thiamin (B1) (mg)", "Riboflavin (B2) (mg)",
"Niacin (B3) (mg)", "Pantothenic acid (B5) (mg)", "Vitamin B6 (mg)", "Biotin (B7)
(mcg)",
"Folate (B9) (mcg)", "Folic acid (mcg)", "Food Folate (mcg)", "Folate DFE (mcg)",
"Choline (mg)", "Betaine (mg)", "Retinol (mcg)", "Carotene, beta (mcg)",
"Carotene, alpha (mcg)", "Lycopene (mcg)", "Lutein + Zeaxanthin (mcg)",
"Vitamin D2 (ergocalciferol) (mcg)", "Vitamin D3 (cholecalciferol) (mcg)",
"Vitamin D (IU) (IU)", "Vitamin K (mcg)", "Dihydrophyllloquinone (mcg)",
"Menaquinone-4 (mcg)", "Fatty acids, total monounsaturated (mg)",
"Fatty acids, total polyunsaturated (mg)", "18:3 n-3 c,c,c (ALA) (mg)",
"20:5 n-3 (EPA) (mg)", "22:5 n-3 (DPA) (mg)", "22:6 n-3 (DHA) (mg)",
"Tryptophan (mg)", "Threonine (mg)", "Isoleucine (mg)", "Leucine (mg)",
"Lysine (mg)", "Methionine (mg)", "Cystine (mg)", "Phenylalanine (mg)",
```

```

"Tyrosine (mg)", "Valine (mg)", "Arginine (mg)", "Histidine (mg)",
"Alanine (mg)", "Aspartic acid (mg)", "Glutamic acid (mg)", "Glycine (mg)",
"Proline (mg)", "Serine (mg)", "Hydroxyproline (mg)", "Alcohol (g)",
"Caffeine (mg)", "Theobromine (mg)", "Serving Weight 1 (g)",
"Serving Description 1 (g)",
"Serving Weight 2 (g)", "Serving Description 2 (g)",
"Serving Weight 3 (g)", "Serving Description 3 (g)",
"Serving Weight 4 (g)", "Serving Description 4 (g)",
"Serving Weight 5 (g)", "Serving Description 5 (g)",
"Serving Weight 6 (g)", "Serving Description 6 (g)",
"Serving Weight 7 (g)", "Serving Description 7 (g)",
"Serving Weight 8 (g)", "Serving Description 8 (g)",
"Serving Weight 9 (g)", "Serving Description 9 (g)",
"200 Calorie Weight (g)"
], axis = 1, inplace = True)

```

Code explanation :

1. `df = pd.read_excel("/Users/supriyauppala/Desktop/myflask/Mydata.xlsx")` - Making a data frame from a .xlsx file
2. `df = df.loc[df['Food Group'] == 'Fruits']` - Extracting a new data frame from the previous data frame that has only the information of Fruits category.
3. `df.drop(["ID", "Calcium (mg)", "Iron, Fe (mg)", .....])` - Dropping the columns i.e, information that is comparatively insignificant.

Data analysis :

The following is the code snippet that implements the data analysis :

```
d2 = pd.DataFrame({'name':[], 'Food Group':[],

'Calories':[],

'Fat (g)':[],

'Protein (g)':[],

'Carbohydrate (g)':[],

'Sugars (g)':[],

'Fiber (g)':[],

'Cholesterol (mg)':[],

'Saturated Fats (g)':[], 'Water (g)':[]})

names = df['name'].tolist()

for i in range(len(names)):

    d2 = df[(df['name']==names[i])]

    d2 = d2.rename(columns={'Food Group':'category', 'Fat (g)':'fat',

'Protein (g)':'protein',

'Carbohydrate (g)':'carbohydrate', 'Sugars (g)':'sugars',

'Fiber (g)':'fibers', 'Cholesterol (mg)':'cholesterol',

'Saturated Fats (g)':'saturatedfats', 'Water (g)':'water'})

    d2["fat"] = d2["fat"].fillna(0)

    d2["protein"] = d2["protein"].fillna(0)

    d2["carbohydrate"] = d2["carbohydrate"].fillna(0)

    d2["sugars"] = d2["sugars"].fillna(0)

    d2["fibers"] = d2["fibers"].fillna(0)
```

```
d2["cholesterol"] = d2["cholesterol"].fillna(0)

d2["saturatedfats"] = d2["saturatedfats"].fillna(0)

d2["water"] = d2["water"].fillna(0)

my_list=[]

for rows in d2.itertuples():

    my_list=[rows.name, rows.category, rows.Calories,

            rows.fat, rows.protein, rows.carbohydrate, rows.sugars ,

            rows.fibers , rows.cholesterol , rows.saturatedfats, rows.water]

new_list=[]

for j in range(3,len(my_list)):

    new_list.append(my_list[j])

labels = ['Fats', 'Proteins', 'Carbohydrates', 'Sugars', 'Fibers',

          'Cholesterols', 'Saturated fats', 'Water']

colors = ['gold', 'yellowgreen', 'lightcoral',

          'lightskyblue', 'mediumpurple', 'black', 'brown', 'orangered']

matplotlib.pyplot.title(label=names[i])

# Plot

sizes=[]

new_labels=[]

new_colors=[]

for ni in range(0,len(new_list)):

    if float(new_list[ni]) != 0.0:

        new_labels.append(labels[ni])
```

```

        new_colors.append(colors[ni])

        sizes.append(new_list[ni])

    patches, texts = plt.pie(sizes, colors=new_colors, startangle=140)

    plt.legend(patches, new_labels, loc="best")

    plt.axis('equal')

    string = '/' + str(names[i]) + '.'

    for op in range(1, len(string)):

        if string[op] == '/':

            string = string[0:op]

            break

    plt.savefig('/Users/supriyauppala/Desktop/myflask/static/' + string +
'png')

    print("Done making a copy of ", i+1)

```

Code explanation :

1. `names = df['name'].tolist()` - Making a list that has the names of all fruits in the data frame

2. `for` each fruit `in` `fruits` list:

```
d2 = df[(df['name']==names[i])]
```

`#` gives a single row data frame that has the current fruit details

```
for rows in d2.itertuples():
```

```
    my_list=[rows.name, rows.category, rows.Calories,
```

```
            rows.fat, rows.protein, rows.carbohydrate, rows.sugars ,
```

```
rows.fibers , rows.cholesterol , rows.saturatedfats, rows.water]
```

```
# my_list has all the details of the current fruit in list format
```

3.

```
labels = ['Fats', 'Proteins', 'Carbohydrates', 'Sugars', 'Fibers',  
'Cholesterols', 'Saturated fats', 'Water']
```

```
colors = ['gold', 'yellowgreen', 'lightcoral',  
'lightskyblue', 'mediumpurple', 'black', 'brown', 'orangered']
```

```
patches, texts = plt.pie(sizes, colors=new_colors, startangle=140)
```

```
plt.legend(patches, new_labels, loc="best")
```

```
plt.axis('equal')
```

```
plt.savefig('/Users/supriyappala/Desktop/myflask/static/' + string + 'png')
```

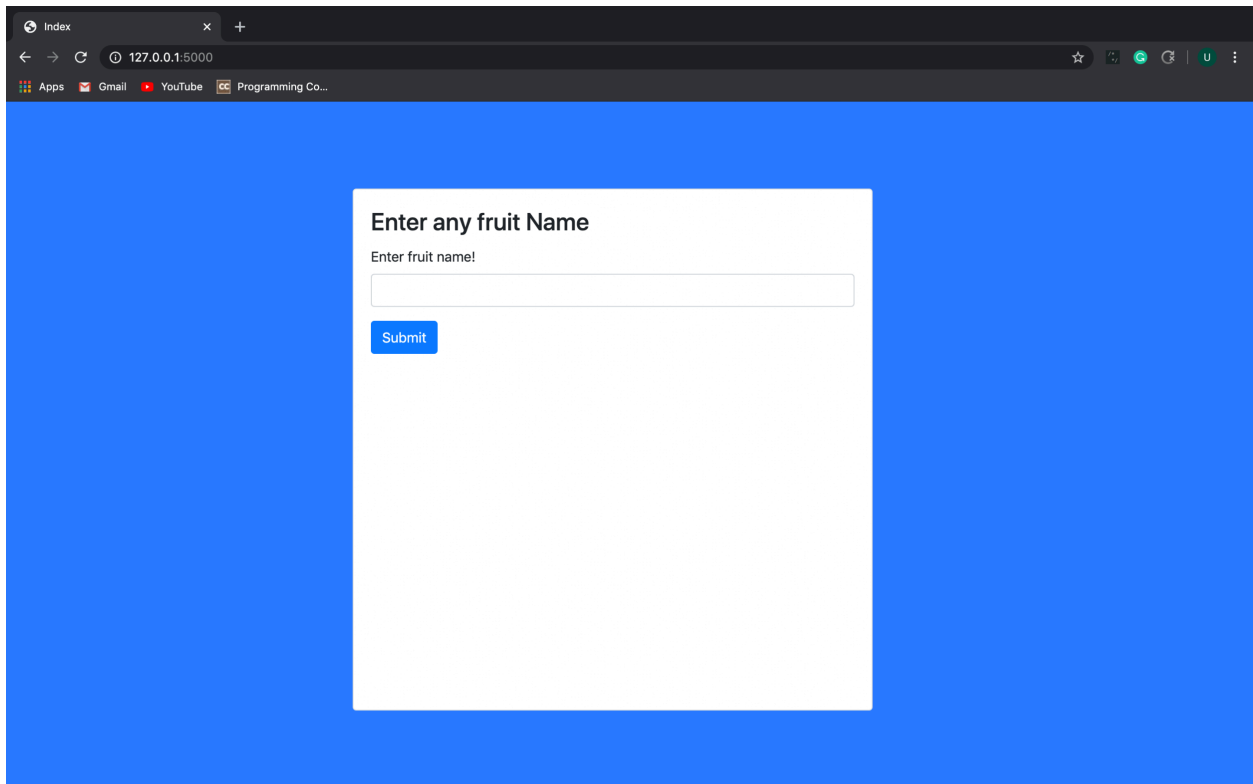
```
# plotting the image and saving the image to the myflask/static directory
```

Because of the loop, all the fruits go into the loop, and their respective images get saved in the directory.

Building the website using Flask

App routing is used to map the specific URL with the associated function that is intended to perform some task. It is used to access some particular page in the web application.

In our application, the URL ('/') is associated with the main function that returns the below HTML page at our website.



In other words, we can say that if we visit the particular URL mapped to some particular function, the output of that function is rendered on the browser's screen.

For each function, a different value is sent as data to the HTML file.

Depending on the data that is sent to the HTML file, the appropriate portions of the HTML are displayed at the site. It is to be noted that for all conditions the same HTML file is returned and rendered.

Below is the application code snippet :

```
from flask import Flask, render_template, url_for, request, redirect

import os

app = Flask(__name__)

@app.route("/")

def main():
```

```
    return render_template('index.html', data='')

@app.route('/enterit', methods = ['POST', 'GET'])

def enterit():

    if request.method == 'POST':

        fn = request.form['nm']

        if len(str(fn)) == 0:

            return render_template('index.html', data="Enter any fruit name!")

        else:

            return redirect(url_for('fruit', name = fn))

    else:

        fn = request.args.get('nm')

        return redirect(url_for('fruit', name = fn))

@app.route('/fruit/<name>')

def fruit(name):

    lst = [list of all fruit names]

    temp = False

    source = ""

    for i in range(len(lst)):

        print(i)

        print(name)

        print(lst[i])

        if lst[i] == name:

            str1 = "../static/"
```

```
        str2 = str(lst[i])

        str3 = ".png"

        source = str1+str2+str3

        temp = True

        break

    if temp == True:

        return render_template('index.html', data=source)

    else:

        return render_template('index.html', data="Not Found")

if __name__ == "__main__":

    app.run(debug=True)
```

Testing

A video containing the implementation of the whole code as well as all the test cases is being attached in the same directory.

Running the application

Flask provides a run command to run the application with a development server.

By the following steps, an application(local) can be initiated :

1. Go to the directory where the application code resides
2. Open terminal in that directory
3. Run the 'flask run' command, the application is run with a development server.

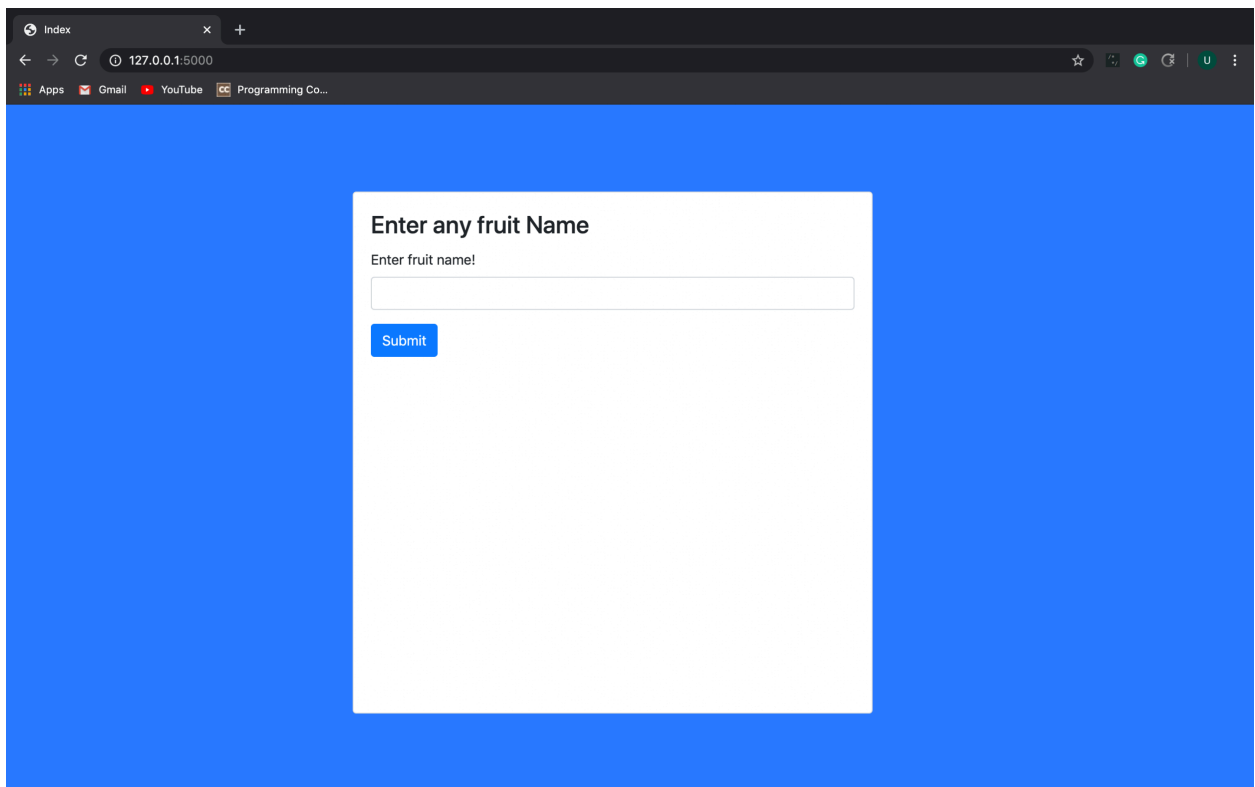
The output looks like this :

```
[Supriyas-MacBook-Air:myflask supriyauppala$ flask run
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

Open the application

Open any browser and enter <http://127.0.0.1:5000/> and click enter

The following page appears in the browser:

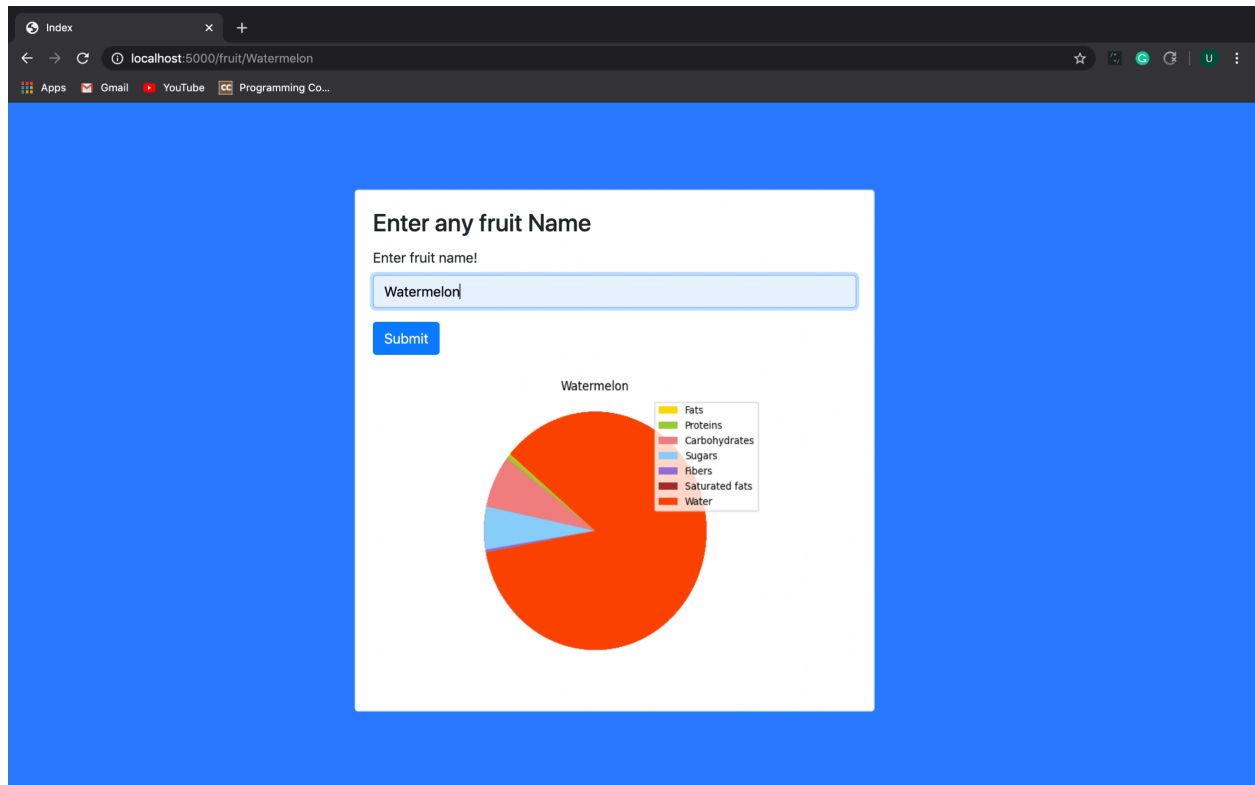


The screenshot shows a web browser window with a single tab titled 'Index'. The address bar displays '127.0.0.1:5000'. The browser's taskbar at the bottom includes icons for 'Apps', 'Gmail', 'YouTube', and 'Programming Co...'. The main content area has a solid blue background. Centered on this background is a white rectangular form. The form has a title 'Enter any fruit Name' in bold. Below the title is a label 'Enter fruit name!' followed by a text input field. At the bottom of the form is a blue button with the text 'Submit'.

Different Test Cases :

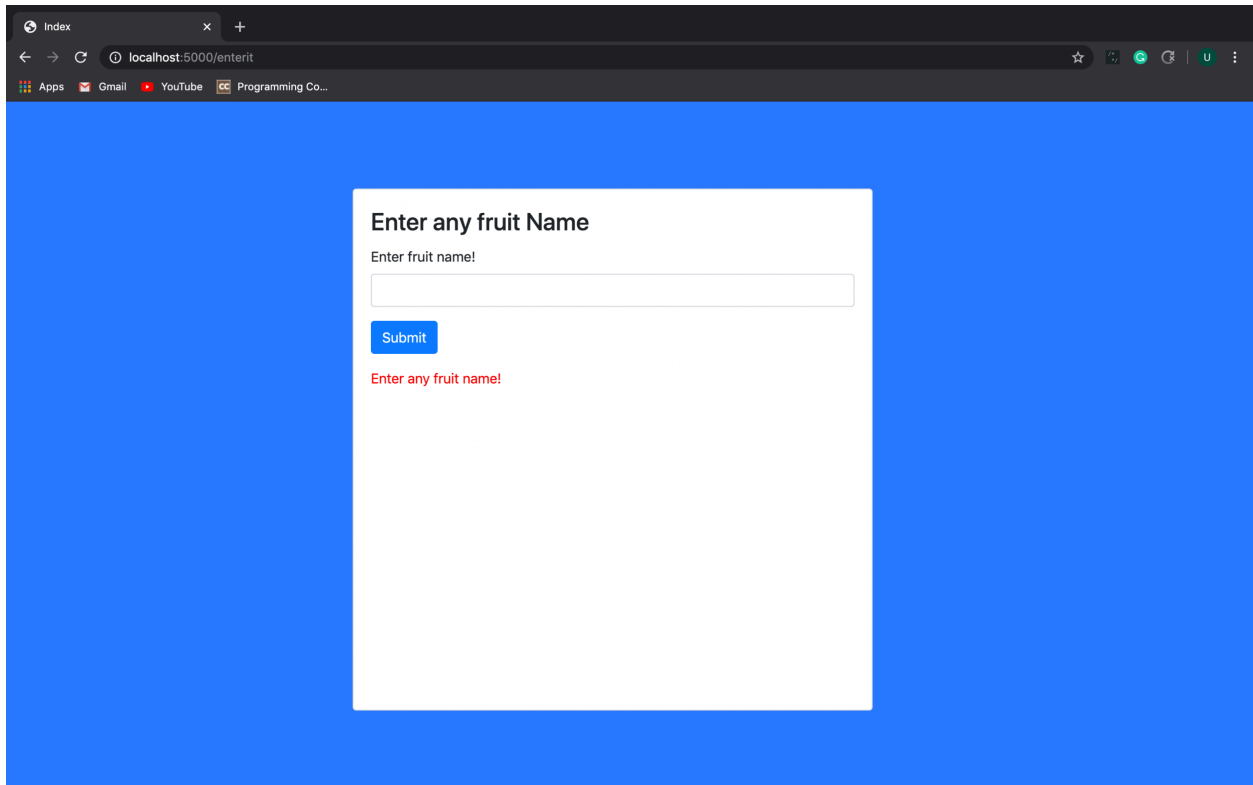
1. If a valid fruit name is entered :

A pie chart containing the nutritional information of the fruit is displayed.



2. If an empty value is given as fruit name :

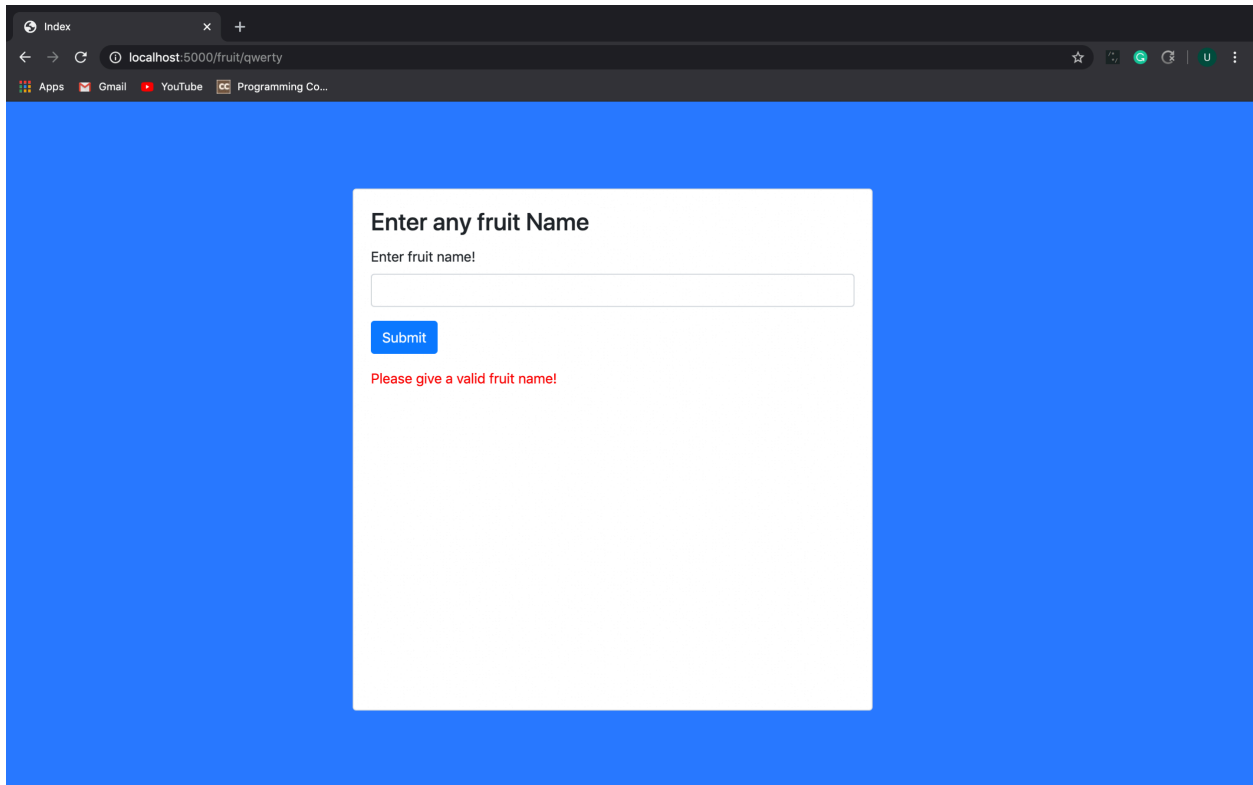
The output looks like this :



The screenshot shows a web browser window with the address bar displaying 'localhost:5000/enterit'. The page has a solid blue background. In the center, there is a white rectangular form. The form has a title 'Enter any fruit Name' in bold black text. Below the title is a label 'Enter fruit name!' followed by an empty text input field. Underneath the input field is a blue 'Submit' button. Below the button, there is a red error message that reads 'Enter any fruit name!'.

3. If invalid input is given as fruit name :

The output looks like this :



Conclusion

This project was a combination of two important fields - Data analysis and Website development. I strictly restricted myself to using only Python and concepts that were taught in the 7 day bootcamp. I understood the problems that arise when unifying two fields of technology together and how they can be overcome by constantly learning and developing.

It was a wonderful learning experience for me while working on this project. This project took me through the various phases of project development and gave me real insight into the world of frontend and backend development. The joy of working and the thrill involved while tackling the various problems and challenges gave me a feel of the developers' industry. It was due to this project I came to know how an error is to be rectified by being ready to explore and learn new things.

Advantages

The main pros of this website are :

- 1) Simple User Interface that allows any layman to easily understand how to proceed
- 2) Not going to a complicated analysis of the nutritional value with values, percentages, etc and a detailed text-based description, a pie chart is the best and most suitable way.
- 3) Pie charts are visually simpler than other charts and they need a minimal additional explanation.

Future Work

Expansion to a wide variety of food :

A future-proof website is not only beautiful but also highly functional. The very next future plan is to expand our website which for now is related to Fruits to Vegetables, packed food, fast foods, restaurant food, etc.

Very Next move :

The website, for now, gives only the nutritional information plotted as a pie chart, but this basic idea can diverge into many ideas:

- 1) Calorie values of the food
- 2) Nutritional values (a chart or a description) of the food they consume in a meal by allowing the users to give the ingredients and the amount of the ingredient they use to make the meal.
- 3) Ranking the food in the database based on the amount of Calcium, Iron, etc so people with deficiencies can include this food in their diets.
- 4) Giving the user a balanced diet description/menu for the day (or the best diet possible) by allowing the users to give the ingredients or items available at their home.

Front end development :

User Interface Design is one of the reasons that a website starts to see an influx of traffic. It draws people in and keeps them there. It is what makes people recommend a site and become loyal customers. Regardless of what anyone says, it should never be overlooked.

Host the website :

Hosting the website on any platform

Acknowledgement

I am highly indebted to Codegnan tutors for their guidance and for providing necessary information regarding the project and also for their support in completing the project.

I would like to express my gratitude towards the members of Codegnan for their kind cooperation and encouragement which help me in completion of this project.

References

- 1) <https://drive.google.com/drive/folders/1C5ThhPLhDvonVJRwfZrS16gPYsghEtt7?usp=sharing>
- 2) <https://www.youtube.com/watch?v=ossvjB4yK14>
- 3) <https://drive.google.com/drive/folders/1jAggQorU3DoiFrouRomw08Go6gyZPlyw?usp=sharing>
- 4) <https://www.youtube.com/watch?v=jIKpPwL8XhM>
- 5) <https://tinyurl.com/notes4qna>
- 6) <https://www.youtube.com/watch?v=x6HcpLhU0C4>